

Lab: Azure App Configuration – Feature Flags

To Begin with the Lab

Summary

Azure App Configuration Feature Flags allow developers to enable or disable application features without modifying or redeploying the application. Feature flags support gradual rollouts, A/B testing, and environment-specific functionality, making application updates safer and more flexible.

Understanding

Feature Flags are configuration settings that control whether a feature is enabled or disabled at runtime. Instead of changing application code, developers simply update the feature flag in Azure App Configuration. Applications check the flag value and decide whether to display or execute the feature. This enables teams to release features gradually, test new functionality with selected users, or quickly disable a problematic feature without redeploying the application.

Example:

Suppose an online shopping application introduces a **Dark Mode** feature.

Development Environment

- Feature Flag: DarkMode = Enabled

Production Environment

- Feature Flag: DarkMode = Disabled

Developers can test the feature in Development while Production users continue using the existing interface. Once testing is complete, enabling the feature only requires changing the feature flag in Azure App Configuration.

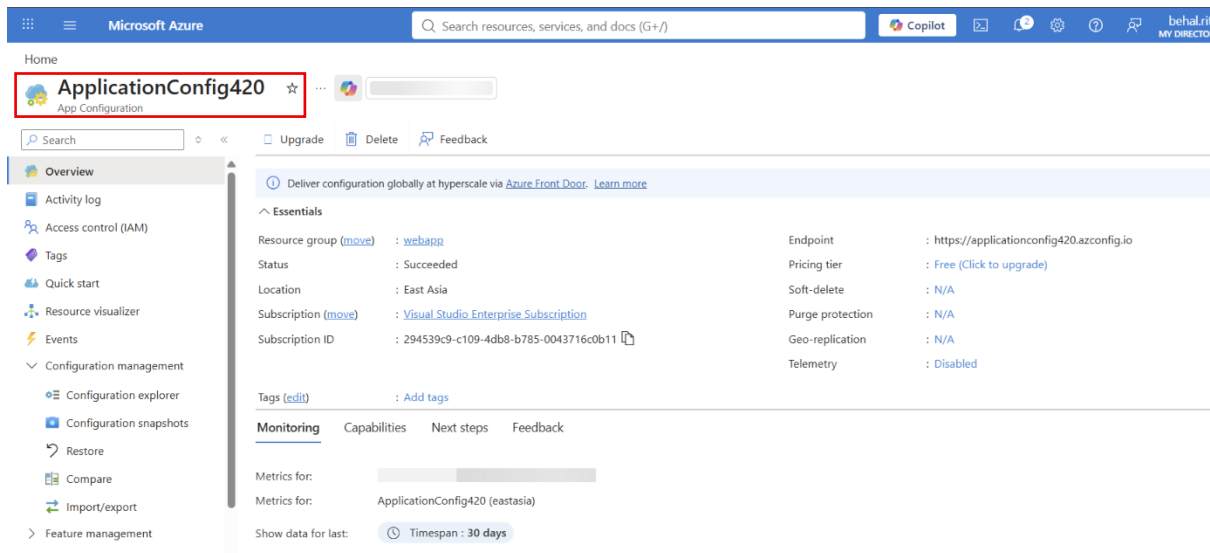
Lab Steps

Step 1: Sign in to Azure Portal

- Open the Azure Portal.
- Sign in with your Azure account.

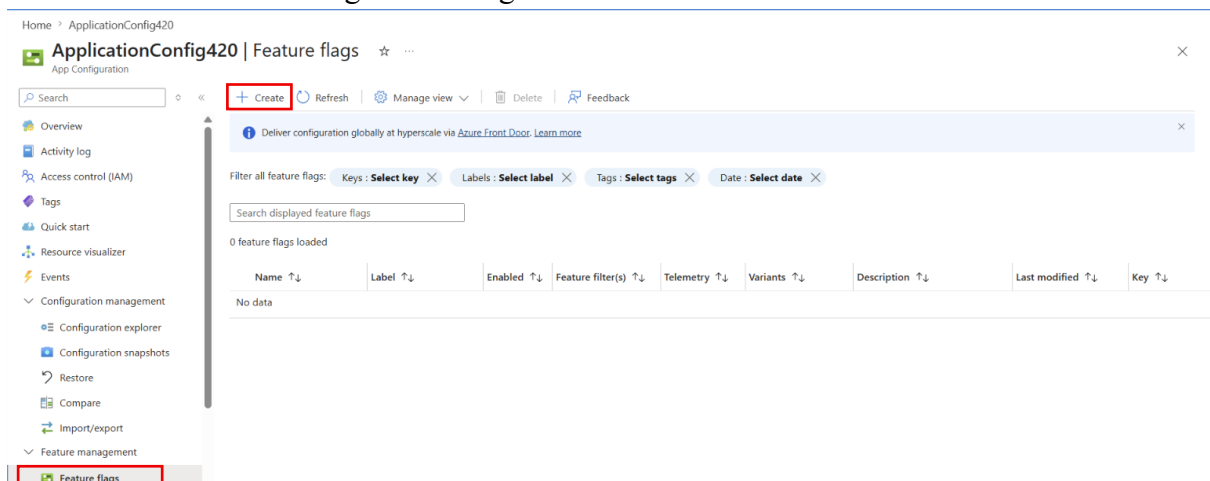
Step 2: Open Azure App Configuration

- Search for **App Configuration**.
- Select your App Configuration resource.



Step 3: Open Feature Manager

- Click **+ Create**.
- Click **Feature Manager** from the left menu.
- View the list of existing feature flags.



Step 4: Create a New Feature Flag

- Enter a unique Feature Flag name.
- Select a Label if required but i have no required.
- Leave the State as Disabled or enable it as needed.
- Click **Review + Create**.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > ApplicationConfig420 | Feature flags

Create

...

Create a new feature flag

What will you be using your feature flag for?

Switch

Rollout

Experiment

Toggle features on and off with a simple switch

Basics

Telemetry

Review + create

Enable feature flag

Details

Name *

ShowDiscountBanner

Key

.appconfig.featureflag/ShowDiscountBanner

Label

(No label)

Description

Description

Review + create

< Previous

Next >

Discard

- Click on the Create

Create

...

Create a new feature flag

Validation passed

What will you be using your feature flag for?

Switch

Rollout

Experiment

Toggle features on and off with a simple switch

Basics

Telemetry

Review + create

Details

Name

ShowDiscountBanner

Label

(No label)

Key

.appconfig.featureflag/ShowDiscountBanner

Description

None

Enabled

False

Telemetry

Telemetry

Disabled

Create

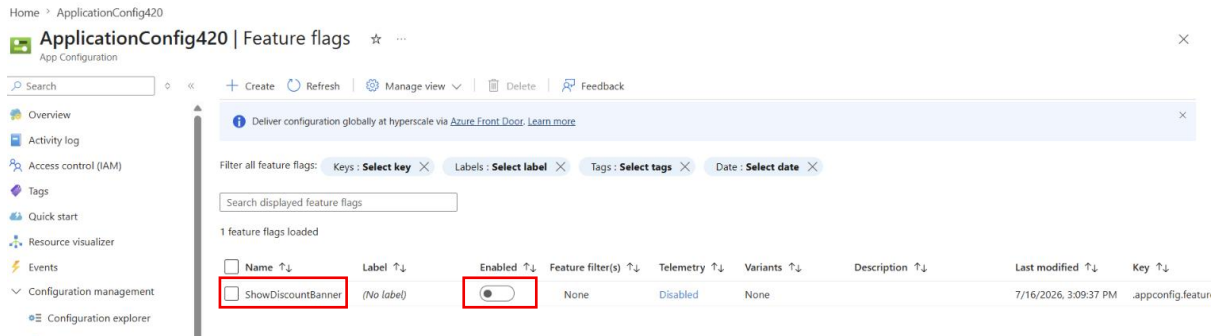
< Previous

Next >

Discard

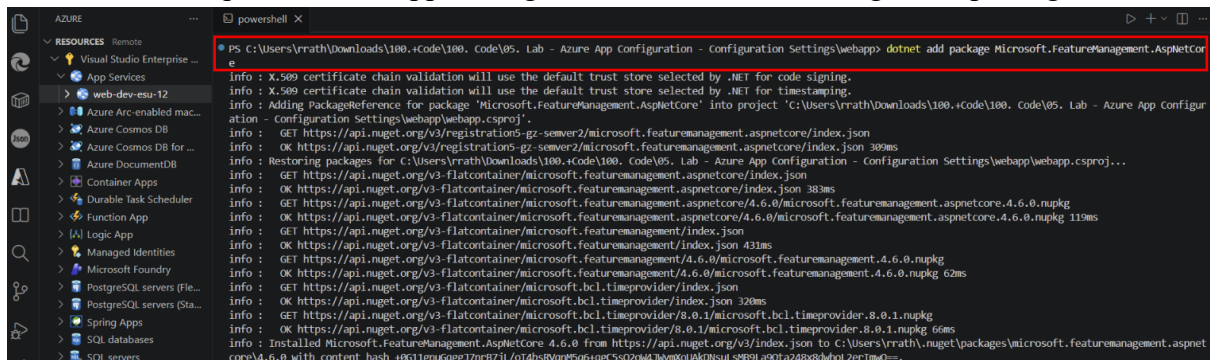
Step 5: Configure Feature Flag

- Open the created feature flag.
- Feature Flag is **Disabled**



Step 6: Connect the Application

- Open the project.
- Install the required Azure App Configuration and Feature Management packages.



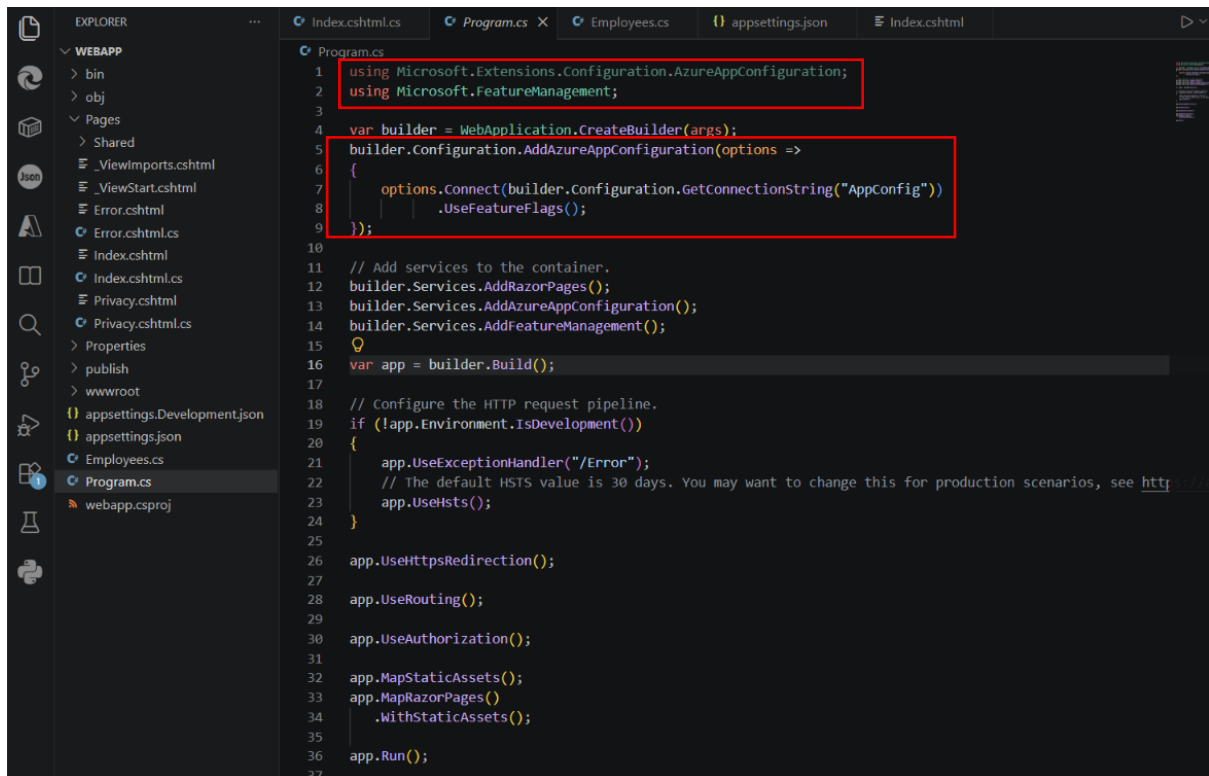
- Go to program.cs file, include Feature flag

using Microsoft.Extensions.Configuration.AzureAppConfiguration;
using Microsoft.FeatureManagement;

- Configure the application to load feature flags.

```
builder.Configuration.AddAzureAppConfiguration(options =>
{
    options.Connect(builder.Configuration.GetConnectionString("AppConfig"))
        .UseFeatureFlags();
});
```

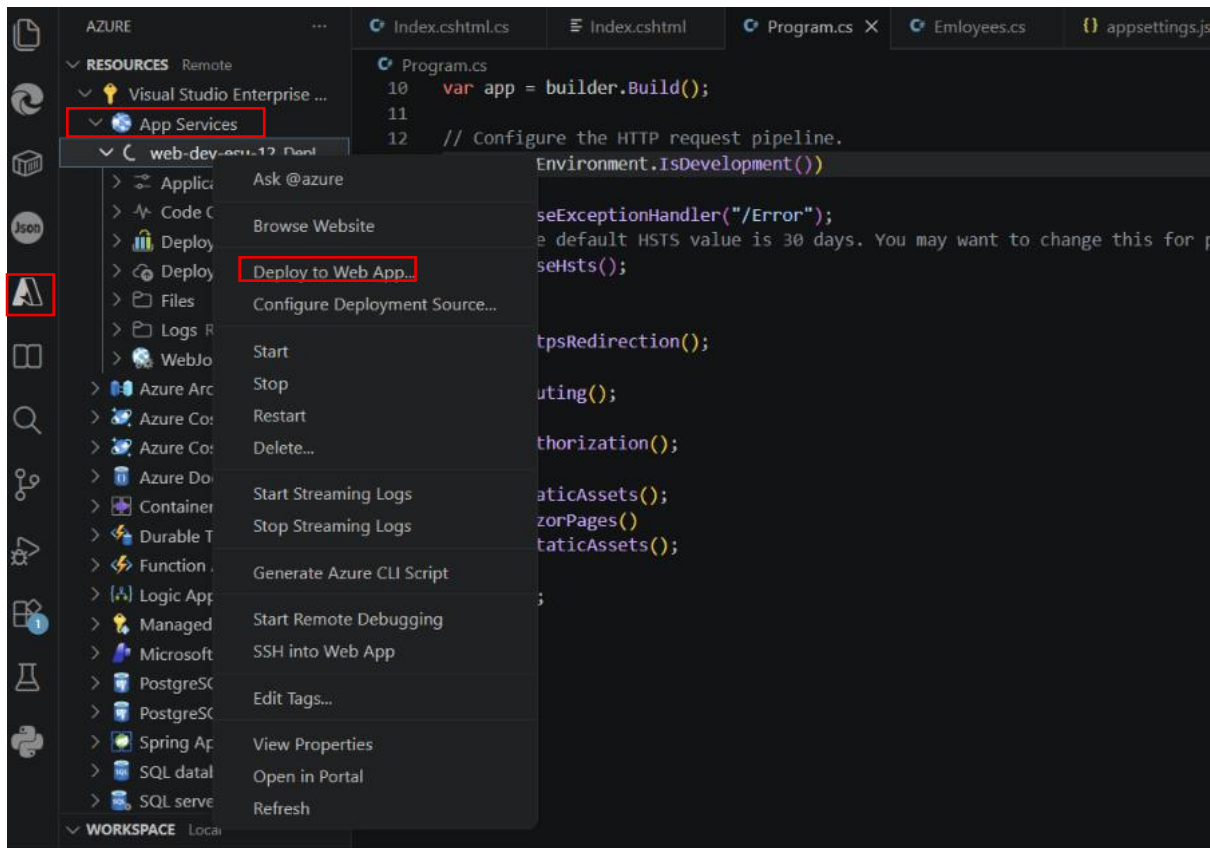
Remove BannerMessage in Files



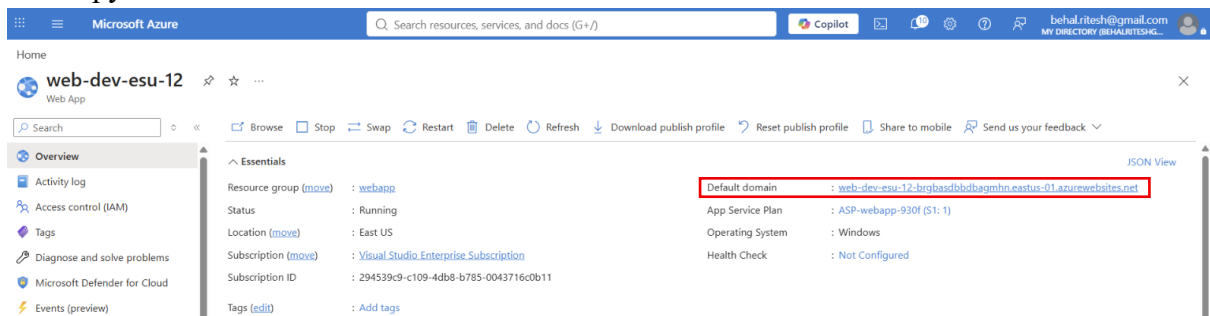
```
1 using Microsoft.Extensions.Configuration.AzureAppConfiguration;
2 using Microsoft.FeatureManagement;
3
4 var builder = WebApplication.CreateBuilder(args);
5 builder.Configuration.AddAzureAppConfiguration(options =>
6 {
7     options.Connect(builder.Configuration.GetConnectionString("AppConfig"))
8         .UseFeatureFlags();
9 });
10
11 // Add services to the container.
12 builder.Services.AddRazorPages();
13 builder.Services.AddAzureAppConfiguration();
14 builder.Services.AddFeatureManagement();
15
16 var app = builder.Build();
17
18 // Configure the HTTP request pipeline.
19 if (!app.Environment.IsDevelopment())
20 {
21     app.UseExceptionHandler("/Error");
22     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
23     app.UseHsts();
24 }
25
26 app.UseHttpsRedirection();
27
28 app.UseRouting();
29
30 app.UseAuthorization();
31
32 app.MapStaticAssets();
33 app.MapRazorPages()
34     .WithStaticAssets();
35
36 app.Run();
37
```

Step 7: Publish the WebApp

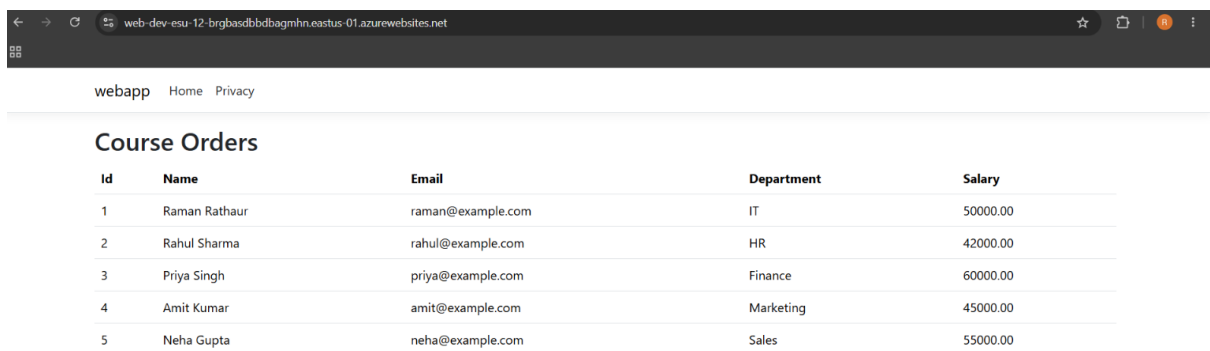
- Go to the Azure in VS code
- Navigate to **App Service**
- **Right-click** on the WebApp
- Click on the **Deploy**
- Wait until a few minutes



- Go to the own Webapp in Azure
- Copy the Default Domain



- Paste the Browser



Step 8: Enable the Feature Flag

- Return to **Feature Manager**.
- **Enable** the Feature flag.

Home > ApplicationConfig420

ApplicationConfig420 | Feature flags ☆ ...

App Configuration

Search

+ Create Refresh Manage view Delete Feedback

Deliver configuration globally at hyperscale via [Azure Front Door](#) [Learn more](#)

Filter all feature flags: Keys: **Select key** Labels: **Select label** Tags: **Select tags** Date: **Select date**

Search displayed feature flags

1 feature flags loaded

Name ↑↓	Label ↑↓	Enabled ↑↓	Feature filter(s) ↑↓	Telemetry ↑↓	Variants ↑↓	Description ↑↓	Last modified ↑↓	Key ↑↓
ShowDiscountBanner	(No label)	☒	None	Disabled	None		7/16/2026, 4:11:53 PM	appconfig.feat.

- Return to **Azure in VS code**.
- Again Publish Webapp
- Refresh the Webpage.
- Add the Feature Flag

webapp Home Privacy

Course Orders

Discount Week: Use code CLOUDXEUS for 20% off!

Id	Name	Email	Department	Salary
1	Raman Rathaur	raman@example.com	IT	50000.00
2	Rahul Sharma	rahul@example.com	HR	42000.00
3	Priya Singh	priya@example.com	Finance	60000.00
4	Amit Kumar	amit@example.com	Marketing	45000.00
5	Neha Gupta	neha@example.com	Sales	55000.00